Introduction

This project is a quantitative comparison of the behaviors of different vehicles as requested by the U.S. Department of Transportation. This comparison will consist of autonomous vehicles, non-automated passenger vehicles, and pedestrians, in order to thoroughly evaluate the claim that autonomous vehicles (AVs) are safer than other methods of transportation. This analysis will be performed using data from the Third Generation Simulation (TGSIM), approximately 200 seconds of recorded trajectory data.

The data that is available consists of a unique identifier for each agent, whether AV, other vehicle, or pedestrian. Additionally, there are the most common agent types for the trajectory. Both values are represented as integers, as they are mutually exclusive. The data also includes the time in seconds and lane or segment polygon ID where the agent is located. There is the position of the agent represented as a y-location in meters and an x-location in meters, as well as speed and acceleration with both x and y aspects. The data also includes a relative size of each agent, which is represented by a smoothed lateral width and a smoothed longitudinal length. These values can be given as floats, since they correspond to direct size, position, velocity, and acceleration.

Methods

With the data provided by the U.S. Department of Transportation through the Third Generation Simulation Dataset, we analyzed the data with Python and various libraries. The libraries that we used were Pandas, NumPy, Matplotlib, and Seaborn. We used Pandas in order to read in our data set into our Python code in the form of a CSV file.

Understanding the data begins with cleaning and organizing the data set we were given. This involves checking and identifying missing values, removing them from the data, and then the imputation of missing values. This was accomplished by creating a separate variable for the null values that were identified. We then used the Pandas tool ".dropna" to remove the null values. Our imputation process used a similar Pandas feature, ".fillna", to replace all missing values with zero. Our cleaned data was then exported to a new CSV file that we continued to use for the rest of the process.

Next, we converted all time, speed, and acceleration values from floats to integer values. This uses tools in both NumPy and Pandas that identify the type of value and can re-assign the data to be a different type of value.

We imported several additional libraries to support our polygon data, including JSON, Ast, Re, Os, Math, Statistics and Path imported from Pathlib. The code opens a Jupyter Notebook file, and reads in all data as JSON data. We took the data from the dictionary that was labeled "polygons" and converted it into a Python dictionary. We found the ranges and centroids of our

polygons using the Math and Statistics libraries. This was done by finding the minimum x and y values for each polygon, and finding the average of the vertex coordinates. The values are then stored under the variable “poly_ranges”, which contributes to the bounding box of all the polygons.

The data was then sorted based on the type of value and number of unique values. Then a “mapping” variable was created that contained values for AVs, non-AVs, and pedestrians.

Next, we found the speed and acceleration magnitudes using the x and y values for speed and acceleration. A “fast” indicator was created, based on a fast threshold of -0.5. If the item had a greater speed magnitude, it was then considered to be fast.

The polygon data was then converted to NumPy arrays, and the bounding box dimensions were calculated based on the corresponding coordinates. This was to create a scatter plot that represents different objects. All tracked points are represented in gray and overlayed with different object categories. Polygon bounding boxes are represented in red.

Another scatterplot was made with similar methods to highlight the selected categories using larger markers and edges with higher contrast. The other categories were plotted with the smaller markers.

The next steps were to add a speed column to our dataset. Additionally, we added a tag point column that informs the user of points made within the polygon.

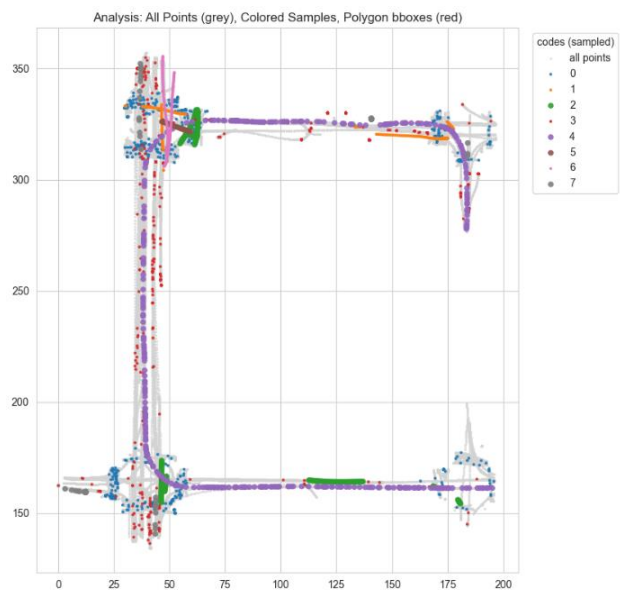
In order to find instances of harsh events within our data, we created a harsh threshold. The harsh threshold value is -0.5, and if an object has a deceleration value greater than the harsh threshold, it will return whether the object was an AV, non-AV, or pedestrian. This way, instances of severe braking will be sorted based on the types of vehicles involved.

For our last figure, we created a boxplot that grouped the data by vehicle size and calculated summary statistics for peak deceleration, including count, mean, median, and standard deviation. We then created a boxplot to compare the distribution of peak deceleration across small, medium, and large vehicles. A horizontal reference line at zero was added to help interpret positive and negative values.

Results and Discussion

The following plots provide a spatial analysis of the trajectory data and polygon regions within the dataset.

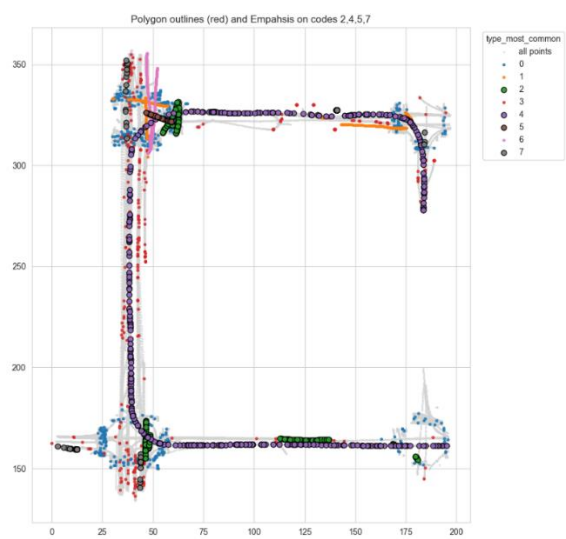
Figure 1



Trajectory points with polygon bounding boxes, showing spatial representation of the dataset.

Figure 1 plots all points in gray, shows sampled categories in different colors, and adds red rectangular bounding boxes around each polygon. The clustered areas show that many roadway events occur at intersections, while straight roads tend to have more consistency in vehicle behavior.

Figure 2

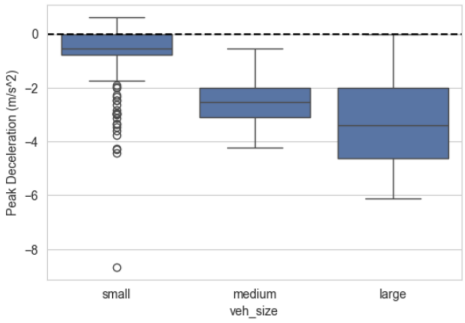


Polygon outlines with sampled trajectory points, highlighting object categories 2, 4, 5, and 7.

Figure 2 shows similarly to Figure 1, that codes 2, 4, 5, and 7 tend to cluster in areas with intersections, while largely remaining constant along straight paths.

Figure 3

veh_size	count	mean	median	std
small	399	-0.614670	-0.538474	0.840748
medium	18	-2.558304	-2.537304	0.873826
large	123	-3.229809	-3.396431	1.513224



Boxplot comparing peak deceleration across small, medium, and large vehicles.

Figure 3 shows a correlation between the size of vehicles and peak deceleration. Larger vehicles have higher deceleration values but also have more variability within the sample size. The small vehicles generally have lower deceleration values but also have many outliers within the data set.

Conclusions

This project analyzed spatial data and vehicle behavior using plots and summary statistics. The spatial analysis showed where data points are located relative to polygon regions and how object types are distributed. The vehicle analysis compared peak deceleration across small, medium, and large vehicles. Overall, the results summarize spatial patterns and differences in braking behavior in the dataset.